Dynamic Resource Task Negotiation to Enable Product Agent Exploration in Multi-Agent Manufacturing Systems

Ilya Kovalenko, Daria Ryashentseva, Birgit Vogel-Heuser, Dawn Tilbury, and Kira Barton

***Abstract*—Distributed multi-agent control has been proposed to improve the flexibility of manufacturing systems. Two important components of this control strategy are product agents, representing physical parts in the system, and resource agents, representing the system resources. Product agents request actions from resource agents to guide physical parts through the manufacturing system. Before requesting specific actions, a product agent must effectively communicate with the system's resource agents to understand the capabilities of the resources on the plant floor. In this letter, an approach to product agent exploration based on a network of resource agents is described and analyzed. In addition, results and insights from an implementation of the proposed exploration methodology in a manufacturing system are presented.**

***Index Terms*—Agent-based systems, intelligent and flexible manufacturing, autonomous agents.**

I. Introduction

TO STAY competitive, manufacturing companies must adapt to continuously changing conditions on the plant floor. The adoption of flexible control strategies can help manufacturers integrate new machines into their system, respond to changing customer demands, and handle unexpected machine faults or breakdowns [1]. Thus, multi-agent control has been proposed to improve system flexibility by enabling intelligent, cooperative products and resources [2], [3].

Multi-agent control architectures have various interacting agents [4], [5]. Two agents that exist in most of the previously proposed architectures are product agents (PAs) and resource agents (RAs). The PAs and RA interactions are displayed in Fig. 1. A PA is responsible for controlling an associated physical part, while an RA provides the high-level control for a resource on the plant floor [5]. Therefore, the performance of the system depends on the effectiveness of the PA and RA cooperation, coordination, and decision making.

Manuscript received February 15, 2019; accepted May 24, 2019. Date of publication June 10, 2019; date of current version June 21, 2019. This work was supported in part by the NSF Graduate Fellowship, a gift from Rockwell Automation, and in part by the Bayerische Forschungsstiftung Scholarship. This letter was recommended for publication by Associate Editor C.-B. Yan and Editor J. Li upon evaluation of the reviewers' comments. *(Corresponding author: Ilya Kovalenko.)*

I. Kovalenko, D. Tilbury, and K. Barton are with the Mechanical Engineering Department, University of Michigan, Ann Arbor, MI 48109 USA (e-mail: ikoval@umich.edu; tilbury@umich.edu; bartonkl@umich.edu).

D. Ryashentseva and B. Vogel-Heuser are with the Institute of Automation and Information Systems, Technical University of Munich, 85748 Garching, Germany (e-mail: daria.ryashentseva@tum.de; vogel-heuser@tum.de).

Digital Object Identifier 10.1109/LRA.2019.2921947

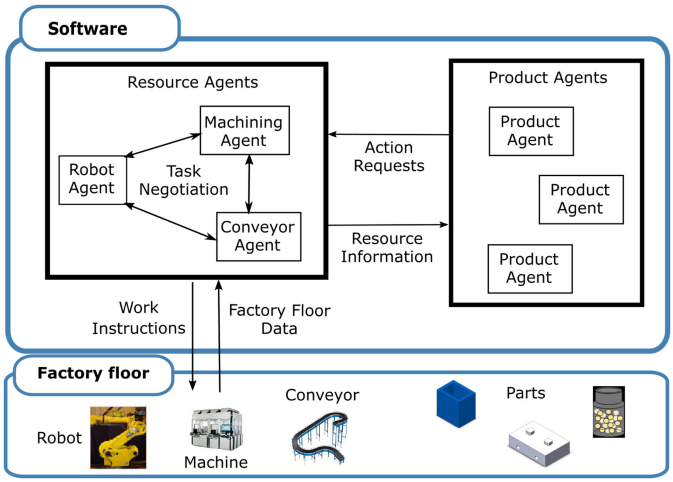


Fig. 1. The interactions between product agents, resource agents, and the factory floor in a multi-agent architecture.

For multi-agent control of manufacturing, a PA is tasked with making various intelligent decisions, such as requesting an RA to execute a manufacturing process or to schedule future operations. Prior to making this decision, a PA must understand the state of the physical part and the capabilities of the surrounding resources. Thus, the PA should be able to dynamically explore the capabilities of the manufacturing environment through communication with the RAs. In existing architectures [6]–[8], PA exploration is accomplished by either supplying the PA with a holistic view of system capabilities or by allowing the PA to query all of the system RAs. Both of these techniques provide the PA with too much information, either creating unnecessary communication overhead or populating the PA knowledge base with extraneous information. Therefore, the contribution of this letter is a novel methodology to enable PA exploration based on a dynamic network of RAs. In the proposed methodology, RAs are able to coordinate and form teams to enable efficient PA exploration.

The rest of the letter is organized as follows. Section II describes the limitations of existing PA exploration techniques. Section III provides the set-up for the new PA exploration technique. Section IV explains how a network of RAs can enable PA exploration and Section V provides attributes of this exploration approach. An implementation is described in Section VI and conclusions are stated in Section VII.

2377-3766 © 2019 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

II. Background

In existing multi-agent architectures, the PA obtains the manufacturing system capabilities by 1) utilizing a global view of the system or by 2) querying all of the system RAs.

A global view of system capabilities has been used to help PAs make decisions in some agent architectures [6], [9]. In these papers, the PA is provided a directed graph of all manufacturing capabilties. The graph may be dynamically generated and is used to make planning decisions [10]. In [6], a PA software architecture to use the graph is proposed. A method to detect changes in the amount of time needed to perform RA actions and update the graph is discussed in [9]. For these implementations, the PA obtains information that is irrelevant to its decision making process, adding unnecessary complexity when finding optimal actions.

Another method for PA exploration is to send operation requests to all RAs in the system, as described in [11]. For example, in [7], [8], [12], the PAs contact all of the RAs in a manufacturing system, allowing the RAs to formulate bids to respond to PA desires. A downside of allowing the PA to contact all of the system RAs is the large amount of communication overhead. Thus, architectures that utilize this type of approach have to manually design a filter that prevents the PA from sending infeasible requests to the RAs. For example, in [8], material handling is ignored to limit the amount of communication overhead. In [13], the resources are split into zones by their location in the manufacturing system. The PA sends a request only to the RAs that control the resources in the same communication zone as the PA's associated physical part. The PA will not be able to find the RAs that are outside the communication zone and, therefore, might not find all of the feasible sequences of actions that will take the physical part to the next desired state.

Both exploration techniques run into problems when ensuring the flexibility of larger manufacturing systems. A single model for the entire system grows in size and complexity, making it difficult to dynamically update all of the individual states and events. Similarly, the communication overhead necessary to negotiate with all of the RAs increases in size and complexity if a PA tries to communicate with all of the system RAs. However, the PA does not need to know about the status of all of the system resources to make intelligent decisions. The need to store a single model or to communicate with all of the system RAs can be prevented by leveraging the structure of the manufacturing system. Therefore, this letter proposes a technique that uses a connected network of RAs. The exploration technique proposed in this letter inherently incorporates dynamic changes to the system, improves PA exploration efficiency, and removes unnecessary PA knowledge and communication overhead.

III. Multi-Agent Architecture

In this section, the agent knowledge needed for exploration via a dynamic resource network is described. The structures used by the PAs and RAs for communication are provided.

A. Resource Agent Knowledge

To utilize the proposed PA exploration, RAs are provided a model of their capabilities and the states that are shared with neighboring RAs.

1) Capabilities Model: A finite state-machine can be used by an RA to represent its resource capabilities [6]. To enable effective communication and cooperation, a resource capabilities model for one RA is defined similarly to the PA belief models described in [13], [14]. The resource capabilities model is defined as the following tuple:

$M = (X, E, Tr, Prp_p, Prp_{np}, T, T_c, x_i, X_m)$, where:

$X = \{x_0, \ldots, x_n\}$: a set of states that can be achieved by any physical part utilizing the resource
$E = \{e_0, \ldots, e_l\}$: a set of events that the RA can trigger that correspond to processes in the physical system
$Tr : X \times E \to X$: a state transition function
$Prp_p : X \to P_p$: a function that maps states to P_p, which is the set of physical properties that represent a change in the physical composition of the part
$Prp_{np} : X \to P_{np}$: a function that maps states to P_{np}, which is the set of physical properties that represent no changes to the physical composition of the part
$T : E \to \mathbb{R}_+$: the time it takes for the event to occur
$T_c : E \to Sc_e$: the current schedule for each event
$x_i \in X$: an initial state (updated for every bid request)
$X_m \subseteq X$: a set of marked states (updated for every bid request)

Events can be logistic or manufacturing operations [15]. An example element of Prp_p is an addition of a feature to a part [16]. Example elements of Prp_{np} are the location and orientation of a part. $Sc_e = \{(t_0, t_1), (t_2, t_3), \ldots, (t_{k-1}, t_k)\}$ represents times when a resource is available for event e, with $0 \leq t_0 < t_1 < t_2 < \ldots < t_k$. Thus, the event can be scheduled between t_0 and t_1, between t_2 and t_3, etc.

This capabilities model is constantly updated as the RA receives information from the resources on the shop floor and other agents in the system. T is updated based on the RA's estimation of event duration. The estimation can be improved with data from the manufacturing system. Sc_e is updated as PAs and other agents request to use the associated resource. Note that x_i and X_m are updated based on an incoming bid request, as described in Section IV-B.

2) Neighboring RAs: For cooperation with other RAs, each RA is provided the states, $X_s \in X$, shared between itself and other resources. Note that shared states have to be a part of the RA's capabilities model. For example, a machine RA and a material handling RA might share a state with one physical property: the drop-off location inside the machine. Each RA stores this information in a table that relates the shared states to specific RAs, $N : X_s \to 2^{RA}$, where 2RA denotes a power set of RA and RA is the set all resource agents in the system. All of the RAs in that table, $RA_{neigh} = \{N(x) : x \in X_s\}$, are the neighboring RAs.

3) Resource Agent Knowledge Example: An example description of a resource capabilities model for a mill is provided in Table I. In this scenario, the mill has pocket milling capabilities and has two neighboring robots, which pick up finished parts and place new parts into the mill.

B. Product Agent Knowledge

The PA contains a process plan and product history [14]. The process plan is provided during PA initialization. As for the

TABLE I
AN EXAMPLE OF RESOURCE AGENT KNOWLEDGE

X	{"At mill1", "Milled pocket"}
E	{"Mill pocket", "Set for pickup"}
Tr	Tr("At mill1", "Mill pocket") = "Milled pocket" Tr("Milled pocket", "Set for pickup") = "At mill1"
Prp_p	Prp_p("At mill1") = $\emptyset$, Prp_p("Milled pocket") = {(pocket, 5x5x2 cm, ...)}
Prp_{np}	Prp_{np}("At mill1") = Prp_{np}("Milled pocket") = {(mill1)}
T	T("Mill pocket") = 120 sec, T("Set for pickup") = 5 sec
T_c	T_c("Mill pocket") = {(0,120),(500,620)}, T_c("Set for pickup") = {(120,125),(620,625)}
N	N("At mill1") = {Robot1 Agent, Robot2 Agent}

product history, the PA keeps track of actions that affect the physical part through communication with RAs.

1) Process Plan: The process plan is a representation of a customer order, defined as the following ordered list: $P_d = (P_1, P_2, \dots P_n)$, where $P_1 = \{p_{11}, \dots, p_{1a}\}$, $P_2 = \{p_{21}, \dots, p_{2b}\}, \dots, P_n = \{p_{n1}, \dots, p_{nm}\}$ and p represents a desired physical property. The desired physical property can be a feature added to a part or a specific location in the manufacturing system, i.e. $p \in P_p \cup P_{np}$. In the process plan, all of the physical properties in a prior set must be complete before trying to accomplish a new set of properties. For example, all of the properties in P_1 and P_2 must be completed before trying to accomplish P_3 properties. The goal of the PA is to request actions from RAs that will enable the physical part to accomplish the entire process plan.

2) Product History: The product history is a representation of the current state of the associated physical part. The product history is defined as: $PH = (X_v, Prp_v, RA_c)$, where $X_v = x_0, x_1, \dots, x_m$ is a sequence of achieved states, $Prp_v =: X_v \rightarrow P_p \cup P_{np}$ is a set of completed physical properties, and RA_c is the last RA to inform the PA of a visited state. To keep the PA informed, the RAs match parts to an associated PA via auto-identification technology [17] (e.g. Radio-frequency identification tags). Then, as a resource performs actions on a part, the associated RA sends information to the identified PA to update X_v, Prp_v, and RA_c.

Some more information and examples of the PA models and properties has been discussed previously in [13], [14].

C. Agent Communication

The two structures used for agent communication are bid requests and bids. PAs use the bid request to start the exploration process and receive bids from RAs. In the proposed methodology, the PAs can send multiple bid requests with different desired physical properties to the RAs. As described in Section IV, the bids and bid requests are used by the RAs to enable task negotiation and by the PAs to formulate a better understanding of the local manufacturing environment.

1) Bid Request: A bid request is defined as $BR = (PA, x_c, P_{id}, t_{bound})$, where PA is the requesting PA, x_c is a starting state, P_{id} is a set of incomplete, desired physical properties and t_{bound}: the maximum allowable time to complete P_{id}.

2) Bid: The bid is defined as: $Bid = (Str_e, Str_x, Ag_h, Prp_p, Prp_{pn}, T, T_c)$, where $Str_e = e_0 e_1 \dots e_{n-1}$ is a sequence of events, $Str_x = x_0 x_1 \dots x_n$ is a sequence of states, $Ag_h : e \rightarrow RA$ is an event-agent association function between each event and the RA that performs that event, and Prp_p, Prp_{pn}, T, T_c are defined similarly to the resource capabilities model.

IV. RESOURCE AGENT TASK NEGOTIATION

In this section, the development and utilization of a dynamic network of resource agents to enable PA exploration is described. The PAs use a bidding process to obtain the capabilities of surrounding resources. For this process, the RAs form teams "on-the-fly" to respond to PA requests.

An overview of the PA exploration approach is shown in Fig. 2. The goal of PA exploration is to obtain the capabilities and schedules of the resources in the vicinity of the associated physical part. The first step in the exploration is a bid request from the PA to the last RA to inform the PA of a visited state. The bid is then propagated by the RA to its neighbors until a team of RAs that can satisfy the bid request is found. If a team of RAs is formed, then a bid is submitted to the PA. Otherwise, RAs keep sharing the bid and bid request until the propagation conditions are not satisfied. After waiting for RA responses, the PA synthesizes the bids to obtain the model of the local manufacturing environment.

A. Product Agent Bid Request

The PA must create a bid request to start the exploration process. For the bid request, x_c is the last state in the sequence of visited states from the PA's product history. P_{id} is created by comparing the completed physical properties in the product history to the properties required by the process plan. Finally, the PA picks a reasonable t_{bound} to provide a maximum amount of time for the RAs to finish P_{id}. Note that the PA can vary both the P_{id} and t_{bound} to expand or limit the amount of information it will receive from the RAs [14]. The initial bid request is sent to RA_c, which is the last RA to contact the PA based on the PA's product history. Note that if the PA had just entered the system (i.e. no product history), it will wait until an RA contacts it with some initial information regarding the location of the physical product.

B. Resource Agent Task Negotiation

An RA uses its resource capabilities model to submit a bid to the requesting PA or propagate the bid request to its neighboring resources. To decide whether to submit or propagate the bid, the RA checks if it can satisfy the bid request. Note that the following definitions are used from discrete event system theory [18]. A string is defined as a sequence of events. $\mathcal{L}(M)$, the set of all strings that are allowable in M, denotes the language generated by M. $\mathcal{L}_m(M) \subseteq \mathcal{L}(M)$, the set of strings in $\mathcal{L}(M)$ that end in a marked state, denotes the marked language of M.

The initial state and marked states of the capabilities model are updated using the bid request. The initial state is defined as: $x_i := x_c$. The marked states are defined as the desired states of the bid request:

$$X_m := \{x \in X \mid Prp_p(x) \in P_{id} \text{ or } Prp_{np}(x) \in P_{id}\} \quad (1)$$

An RA can satisfy a bid request if it finds a string, $s_e = e_0 e_1 \dots e_{n-1}$, and states, $s_x = x_0 x_1 \dots x_n$ that satisfy a few

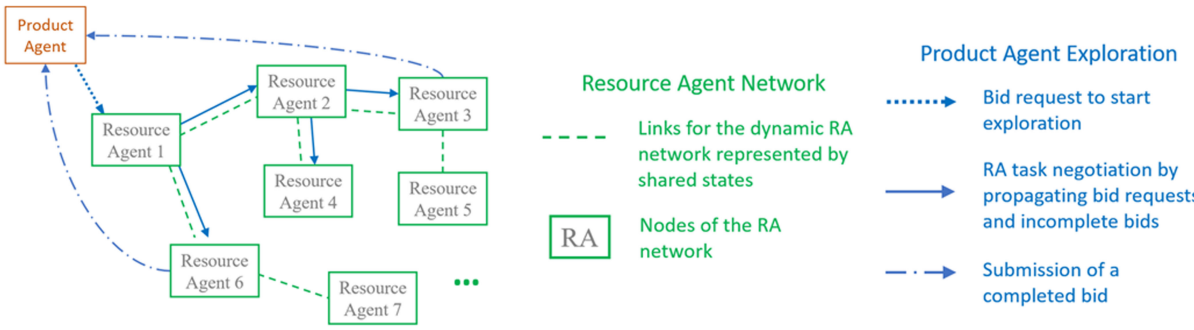


Fig. 2. An overview of the agent communication when the network of RAs is used for PA exploration.

conditions. First, the string must be in the marked language of M and s_x must be a feasible sequence of states in the RA's capabilities model:

$$s_e \in \mathcal{L}_m(M) \text{ and } x_{i+1} = Tr(x_i, e_i), \text{for } 0 \leq i \leq n-1 \quad (2)$$

In addition, all desired physical properties in the PA's bid request must be satisfied. Any physical properties of $x \in s_x$ that alter the part's composition must be a part of the PA's bid request. Thus, the following constraint must be satisfied:

$$\cup_{i=1}^{n} Prp_p(x_i) = P_{id} \setminus \cup_{i=1}^{n} Prp_{np}(x_i) \quad (3)$$

Note that any number of properties that do not alter the part's composition are allowed. However, if the states have extra properties that alter the part's composition, they do not satisfy this constraint. To satisfy Eq. (3), all of the properties in P_{id} must be accomplished in s_x. Finally, all of the properties must be accomplished in the requested amount of time:

$$\sum_{i=1}^{n-1} T(e_i) \leq t_{bound} \quad (4)$$

A bid is submitted if the RA finds s_e and s_x to satisfy these conditions. Otherwise, the bids are propagated to neighbors.

1) Submitting a Bid: If Eq. (2) – (4) are satisfied, the RA creates a bid. For the bid, $Str_e = s_e$ and $Str_x = s_x$ are obtained from the above equations. Prp_p, Prp_{pn}, T, T_c are obtained from the resource capabilities model. For the bid's Ag_h, all of the events in Str_e are mapped to the RA creating the bid. The bid is submitted to the PA in the bid request.

2) Bid Request Propagation: If Eq. (2) – (4) are not satisfied, the RA decides if the bid should be propagated. The marked states are redefined as the set of shared states:

$$X_m := X_s \quad (5)$$

The RA will propagate a bid if there exists a string, $s_{e,p} = e_0 e_1 \dots e_{n-1}$, and states, $s_{x,p} = x_0 x_1 \dots x_n$ that satisfy Eqs. (2) and (4). Note that $s_{e,p}$ has at least one event to ensure propagation, i.e. $n > 0$. Finally, without the requirement to accomplish all of the physical properties in the process plan, Eq. (3) is relaxed. However, the string must not have any physical properties that alter the composition of the part unless part of the desired physical properties:

$$\cup_{i=1}^{n} Prp_p(x_i) \subseteq P_{id} \quad (6)$$

If these conditions are satisfied, the RA creates an incomplete bid, Bid_{ic}, where $Str_{e,ic} = s_{e,p}$, $Str_{x,ic} = s_{x,p}$, $Prp_{p,ic}, Prp_{pn,ic}, T_{ic}, T_{c,ic}$ are obtained from the resource capabilities model, and $Ag_{h,ic}$ maps all of events to the current RA. In addition to Bid_{ic}, a new bid request BR_{ic}, is created in the following manner:

$PA_{ic} = PA$ (no change to the requesting PA)

$$x_{c,ic} = x_n$$

$$P_{id,ic} = P_{id} \setminus \left[\bigcup_{i=1}^{n} \left(Prp_p(x_i) \cup Prp_{np}(x_i)\right)\right]$$

$$t_{bound,ic} = t_{bound} - \sum_{i=1}^{n} T(e_i)$$

where $x_{c,ic}$ is the shared state, $P_{id,ic}$ are the physical properties that must be finished to complete the bid, and $t_{bound,ic}$ is the allowable remaining time to complete those properties. Bid_{ic} and BR_{ic} are sent to $N(x_{c,i})$, the neighboring RAs with the shared state.

3) Creating a Team of Resources: If an RA receives an incomplete bid alongside a bid request, the RA will try to complete the bid by answering the provided bid request. To complete the bid, the RA tries to find a string, $s_{e,c}$, and a sequence of states, $s_{x,c}$, that satisfy 1 – 4 for the propagated bid request, BR_{ic}. If these constraints are satisfied, the RA creates and submits a completed bid. To complete the bid, the state and event sequences are appended to the incomplete bid, i.e. $Str_e = e_{0,ic} \dots e_{n,ic} e_{0,c} \dots e_{m,c}$, where $e_{i,ic} \in Str_{e,ic}$ for $0 \leq i \leq n$, $e_{j,c} \in s_{e,c}$ for $0 \leq j \leq m$. In this formulation, n and m denote the length of the events in the incomplete bid and the newly found sequence of events, respectively. Similarly, the newly found states are appended to the states of the incomplete bid. Using the RA's capabilities, Prp_p, Prp_{pn}, T, T_c are updated based on the newly added states and events. Finally, for Ag_h, all of the added events are mapped to the RA creating the bid. The complete bid is then sent to the PA in the bid request.

If the RA cannot complete the bid, it will propagate the bid further. If Eqs. (2) and (4) – (6) are satisfied by a sequence of events and states, the RA updates the incomplete bid. To update the bid, the state and event sequences are appended to the $Str_{e,ic}$ and $Str_{x,ic}$ and $Prp_{p,ic}, Prp_{pn,ic}, T_{ic}, T_{c,ic}, Ag_{h,ic}$ are updated. The RA sends the updated, but still incomplete, bid and a new bid request to its neighboring RAs.

Algorithm 1: Compiling All Bids Into an Environment Model.

Input: $Bids = Bid_1, Bid_2, \ldots, Bid_m, X_v$
Output: M_e
Initialize: $X_e, E_e, Tr_e, Prp_{p,e}, Prp_{np,e}, Ag_{h,e}, T_e, T_{c,e}$

```
// Iterate through each of the bids:
for all Bid ∈ Bids do
    // Add the states and associated properties
    for all x ∈ Str_{s,Bid} do
        if x ∉ X_e (see Eq. (7) for state equality) then
            Add x to X_e
            Let Prp_{p,e}(x) := Prp_{p,Bid}(x)
            Let Prp_{np,e}(x) := Prp_{np,Bid}(x)
    end for
    // Add the events and associated properties
    for all e_n ∈ Str_{e,Bid} do
    if e_n ∉ E_e (see Eq. (8) for event equality) then
        Add e_n to E_e
        Let Tr_e(x_n, e_n) := x_{n+1}
            where x_n is the n^th element of Str_{s,Bid}
        Let Ag_{h,e}(e) := Ag_{h,Bid}(e)
        Let T_e(e) := T_{Bid}(e)
        Let T_{c,e}(e) := T_{c,Bid}(e)
    end for
end for
Let x_c := x_m, where x_m is the last element in X_v
```

C. Product Agent Bid Compilation

After sending the initial bid request, the PA waits for the RAs to reply with a set of bids. If the PA is not satisfied with the number of bids received (e.g. 0 bids), then it would send out another bid request to re-explore the system. To obtain different bids, the PA can change the number of states that it initially wants completed or increase the maximum allowable time in the bid request.

If the PA is satisfied with the number of bids received, then a model of the manufacturing environment can be compiled [14]. For the PA, the model is the following tuple: $M_e = (X_e, E_e, Tr_e, Prp_{p,e}, Prp_{np,e}, Ag_{h,e}, T_e, T_{c,e}, x_c)$, where x_c is the current state of the physical part, $Ag_{h,e}$ is the event-agent association function, and all other elements are defined similarly to the resource capabilities model. Note that the PA can use this model to determine desired states for the part, as described in [14].

The environment model can be compiled using the fusion operator [19] on the RA bids. Algorithm 1 illustrates how to create M_e using the bids and past product states from the product history, X_v. Note that two states, x_a and x_b, or two events, e_a and e_b, are defined as equivalent if and only if:

$$Prp_p(x_a) = Prp_p(x_b) \text{ and } Prp_{np}(x_a) = Prp_{np}(x_b) \quad (7)$$

$$e_a = e_b \text{ and } Ag_h(e_a) = Ag_h(e_b) \quad (8)$$

In other words, two states are equivalent if they have the same physical properties and two events are equivalent if they have the same name and the same associated RA.

With this newly compiled environment model, the PA can plan, schedule, and request the requisite events from RAs. Before sending scheduling requests, the PA uses this model to understand the RA capabilities and schedules. The PA can find paths to less congested machines and work with the RAs to minimally affect existing RA schedules. In addition, if a PA is not able to find a feasible path after thoroughly exploring the system, the PA will ask for assistance. In the case study described in Section VI, the PA will call a human operator to take the physical part out of the system.

V. Key Attributes of the Proposed Approach

In this section, the dynamic nature of the RA network and the benefits of the proposed approach are discussed.

A. Dynamic Network of Resource Agents

The proposed PA exploration framework is enabled by creating a dynamic communication network of RAs. The nodes in the communication network correspond to individual RAs in the system, while the links represent the shared states between neighboring RAs. Note that a link exists only if both resources have physical access to the shared state.

When a new resource is added into the system, the capabilities model and the table of neighboring resources and states are provided to a newly initialized RA. To establish the link in the communication network, the new resource informs its neighboring RAs about their mutually shared states. Once informed, each neighbor RA can add the new RA to its own table of neighboring resources and states. With the communication link established, bid requests from PAs in the system will be able to reach the new resource agent.

Similarly, an RA can be removed from the communication network (e.g. a resource goes down) by contacting its neighboring RAs. Once the RA is removed from a neighbor's table of shared states, the communication link is broken. Note that another RA might need to take over the removed RA responsibilities if the removal causes a drastic change in the manufacturing system (e.g. unreachable machines).

The connectivity of this network drives the time performance of PA exploration. The speed of PA exploration can be improved when links are removed from the network. While removing links may improve the exploration time performance, RAs can not use the removed connection when responding to PA exploration requests. Thus, the level of connectivity should be optimized based on the capabilities of each resource, the number of PAs and RAs in the system, and the speed of the agent communication and decision making.

B. Benefits of Proposed Negotiation Strategy

1) Flexible Part Behavior: The physical part can be placed anywhere in the system and, if there exists a way to obtain desired physical properties in a reasonable amount of time, the network of resource agents will submit a bid that satisfies the criteria of the associated PA. The PA does not need to have any prior knowledge of the system to function. To obtain an accurate representation of the physical part's environment, the RAs

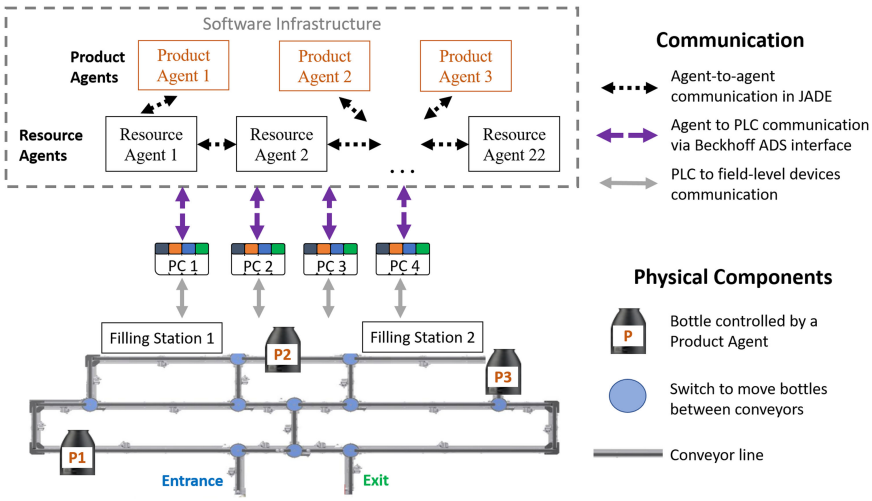


Fig. 3. The setup and infrastructure for the myJoghurt system at the Institute of Automation and Information Systems, Technical University of Munich.

must continuously keep updating the PA regarding the state of the associated physical part.

2) Response to Changes on the Plant Floor: The dynamic network of RAs can help in improving the flexibility of manufacturing systems. The network can capture changes in the capabilities of the entire manufacturing system as RAs are added, moved, or removed. Modifications of individual resource capabilities can also be captured in the RA models. Using the approach presented, PAs can re-explore the system and change their plans to adapt to dynamic conditions on the plant floor or to changes in customer orders.

3) Security of RA Capabilities: The RAs decide how much of their capabilities they share with the PA. The manufacturer can hide some resource capabilities from certain PAs. This provides the manufacturer more control over the amount of shared information between agents. In addition, there is not a central storage place that contains a detailed description of all of the resources. This removes a central point of failure from existing agent-based manufacturing controllers.

4) Integration With Existing Communication Protocols: Due to its bidding mechanics, the proposed technique can be implemented using existing communication protocols. The contract net protocol [11], often used in agent-based manufacturing, was used in the case study for PA exploration.

VI. Implementation and Testing

The concepts of resource agent task negotiation was implemented in the myJoghurt CPPS (Cyber Physical Production System) demonstrator at the Institute of Automation and Information Systems, Technical University of Munich [3], [20]. myJoghurt is a production facility used to test new manufacturing system control technologies for the production of personalized bottles of yogurt based on various customer orders [21]. The demonstrator consists of a bottle storage facility, a logistic system, and two filling stations with different colored pellets (Fig 3). Once a customer places an order for a personalized yogurt production, a bottle is taken from storage and placed into the logistic system. Then it is moved by sets of conveyors to desired locations around the demonstrator. The desired locations for each bottle depend on the customized process plan created using the individual customer order. Thus, individual bottles have different requirements based on their process plans, current locations, deadlines, etc. In addition, the capabilities of the logistic system are always changing due to varying routes of the bottles, conveyor maintenance schedule, or alterations to the sensors and actuators in the system creating a dynamic manufacturing environment.

A. Agent Control and Communication Architecture

The implementation of the multi-agent control architecture for the myJoghurt demonstrator is shown in Fig. 3. The logistic component of the demonstrator contains 22 conveyor lines controlled via stepper motors. Switches route bottles at conveyor junctions. Light sensors indicate the presence of a bottle at each switch and at the beginning and end of each conveyor line. The conveyor motors, switches, and sensors are connected to four embedded PCs (a Beckhoff CX2040 and three Beckhoff CX9020) via EtherCAT. The embedded PCs run Beckhoff TwinCAT3 to control the conveyors, set the direction of the switches, and obtain the presence of bottles at the light switches, among other functions.

The multi-agent control architecture is implemented using the JAVA Agent DEvelopment Framework [22] on a Dell Intel Core i7 PC. 18 RAs are connected to the four embedded PCs using the Beckhoff Automation Device Specifications (ADS) interface [23] over EtherCAT. The communication between agents in the system is accomplished using the FIPA (Foundation for Intelligent Physical Agents) ACL Message protocol in the JADE environment. The ACL Messages contain custom, serialized Java objects that are able to be decoded by each of the agents in the system. The RAs are initialized with individual capabilities models encoded using the JUNG library [24].

The PAs control the behavior of the bottles in the system through communication requests with the RAs. Each PA is provided a process plan with locations around the system and/or the addition of pellets in the first or second filling station. Using

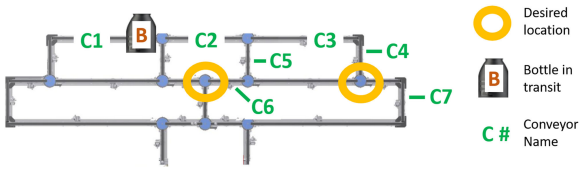


Fig. 4. Setup for an example scenario demonstrating the PA exploration.

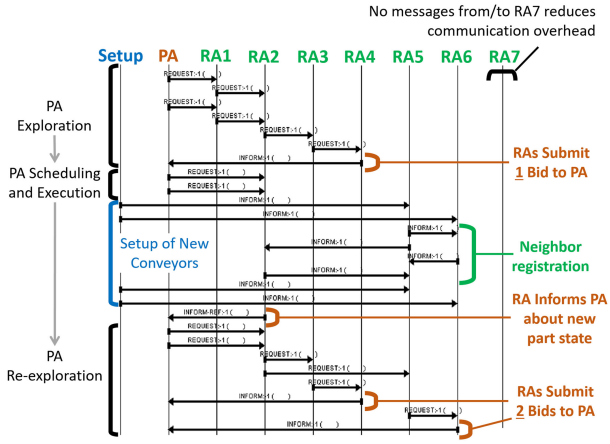


Fig. 5. An screenshot of the communication when a new RA enters the system. The annotations highlight how a PA re-explores the system to learn about this unexpected change.

the process plan, the PAs attempt to explore the system, make plans, and request actions from the RAs.

B. Agent Cooperation

A PA explores the system using the negotiation strategy described in Section IV. Based on the returned bids, the PA can decide whether it needs to further explore the system. If the PA finds multiple paths to a desired state, it will plan to take the shortest time path available. After the PA plans a sequence of actions to take, it executes the action by querying them from the respective RAs.

1) One PA Example: Figs. 4 and 5 show the setup and the communication for one example scenario in the system. In this scenario, the RAs are initially provided with individual capabilities. For example, RA2 is provided a capabilities model that contains the following states: "C2 Start", "C2 End", "C2 to C3", and "C2 to C5". Each of these states is linked to a presence sensor in the system to signify when a bottle arrives at the state. In addition, the events of the capabilities model represent transitions between two states. For example, an event called "Run C2" is a transition between the "C2 Start" and "C2 End" states. These events are linked to PLC tags in the system. Hence, "Run C2' is linked to a tag that starts conveyor 2 and takes a bottle to the end of the line. Finally, the neighboring RAs and the corresponding states are provided to each RA. For example, "C2 to C3' is a neighboring state between RA2 and RA3.

An example of the behavior of a PA in the system is shown in Fig. 5. In this scenario, the bottle is located on conveyor 1 (C1). Based on its process plan, the PA must guide the bottle to one of its two desired locations. First, the PA sends out bid requests with a small allowable time limit. Since the RAs cannot form a team to take the bottle from its current state to a state with a desired location in the allowable time, the PA increases the time limit for its second exploration request. With the new time limit, the team of RA2-RA3-RA4 sends the PA a bid. The PA uses Djikstra's algorithm to plan and schedule an event string to take the bottle from its current state to a desired state in the shortest amount of time [14]. Finally, once the bottle arrives at new states, the RAs inform the PA by monitoring the individual presence sensor.

The dynamic response of the system is also shown in the example. Specifically, C5 and C6 are placed into the system as the bottle is traveling on C1. The RAs associated with those resources enters the network of RAs through communication with their neighbors. Thus, when the PA re-explores the system, the RAs submit two bids that can take the bottle to a desired state. This example shows the flexibility of using the dynamic network of resource agents to quickly adapt to changes in the manufacturing system.

2) Multiple PAs Example: To test the performance of multiple PAs in the system, 20 bottles were sent to the demonstrator. New bottles entered the system every 190 seconds, replicating a realistic production plant. Simulating a random customer order, seven PAs were provided a process plan that required pellets from Filling Stations 1 and 2. The process plans of the other PAs required only Filling Station 1 pellets. The goal of each bottle was to reach the exit of the demonstrator with the correct pellet distribution.

By utilizing the proposed negotiation procedure, the PAs representing the bottles were able to explore, plan, and request actions from the RAs representing the conveyors. Overall, 18 out of 20 bottles successfully navigated the system. 2 of the bottles became stuck due to a mechanical defect in the system. If stuck and unable to finish their process plan, the PAs requested a human operator to remove the associated physical part from the system.

Experiments showed that the RAs required 38 ± 24 (average, standard deviation) milliseconds to perform task negotiation and send back a bid to the PA using JADE's communication infrastructure. The size of submitted bids ranged from 1 to 17 events. The maximum amount of time for a bid to come back to the PA was 125 ms. The exploration time variability was mostly due to message transfer speed in the JADE environment. Once the bids were obtained, the PAs scheduled an action with the RAs. The average time to schedule an event with an RA was 3.2 ms. Finally, the PAs had to request actions from the RAs. On average, it took 19 ± 17 ms, with a maximum of 124 ms, between the time that the PA requested an action to the time when the RAs finished setting the appropriate tag in a Beckhoff PC using the ADS interface. The variability in execution time depended on both the speed of message transfer in the JADE environment and the speed of using the ADS interface.

C. Insights From Implementation

The feasibility of using this exploration technique in a real manufacturing system was shown in this implementation. Exploration was integrated with a controller that uses a common standard for industrial system control (IEC 61131-3). RAs and PAs made timely decisions to control bottles in the system. PA exploration took at most 125 ms with multiple agent

interactions. This time may scale up as the number of agents and the amount of communication in the system increases.

The benefits (Section V-B) are showcased in this implementation. The bottle can be placed in any location with a sensor and will start to explore the system, plan, and execute actions to reach a desired state. The discussed dynamic network of RAs adapts instantly to changes in the manufacturing system. These changes include resource addition, removal or maintenance, or changes in resources capabilities. Note that this exploration technique does not require a single global model or communication with all of the system RAs, as required by alternate exploration techniques. For example, in Fig. 5, RA7 does not have to divulge information about its capabilities to the PA, preventing excess communication. Additionally, each RA updates their own capabilities model, reducing the need to store and update a global system model during PA exploration.

Some of the PA-RA communication parameters have to be tuned for the specific application of an agent-based controller. These parameters include the number of physical properties and the size of the time limit in the PA's bid request, the amount of time the PA needs to wait before it begins to plan and execute actions, and how often the PA must re-explore the system. These parameters will dictate PA robustness and responsiveness and are unique to the manufacturing system. The selection of these parameters will be based on speed of the agent communication infrastructure, fidelity of each resource capabilities model, and expected amount of disturbances.

VII. Conclusion

The multi-agent control strategy consisting of product and resource agents (PAs and RAs) has been proposed to improve manufacturing system flexibility. For this strategy, the PAs must effectively guide an associated physical part through a manufacturing system. To accomplish this goal, PAs must understand the capabilities of the surrounding environment. In this work, a novel approach for PA exploration based on the propagation of bids and bid requests over a dynamic network of RAs is proposed. Using the proposed methodology, RAs form teams to provide PAs with a comprehensive view of the surrounding environment. In addition to enabling RA task negotiation, this approach limits the amount of information available to agents, improving security and reducing communication overhead of agent systems. Future work will include incorporating agent cooperation that guarantees deadlock avoidance, analyzing the RA network connectivity, and optimizing the amount of information transfer.

References

[1] K. Barton, F. Maturana, and D. Tilbury, "Closing the loop in IoT-enabled manufacturing systems: Challenges and opportunities," in *Proc. Annu. Amer. Control Conf.*, 2018, pp. 5503–5509.

[2] P. Leitão and S. Karnouskos, *Industrial Agents: Emerging Applications of Software Agents in Industry*. San Mateo, CA, USA: Morgan Kaufmann, 2015.

[3] B. Vogel-Heuser, C. Diedrich, D. Pantförder, and P. Göhner, "Coupling heterogeneous production systems by a multi-agent based cyber-physical production system," in *Proc. 12th IEEE Int. Conf. Ind. Inform.*, 2014, pp. 713–719.

[4] A. M. Farid and L. Ribeiro, "An axiomatic design of a multiagent reconfigurable mechatronic system architecture," *IEEE Trans. Ind. Inform.*, vol. 11, no. 5, pp. 1142–1155, Oct. 2015.

[5] P. Vrba *et al.*, "Rockwell automation's holonic and multiagent control systems compendium," *IEEE Trans. Syst., Man, Cybern. C, Appl. Rev.*, vol. 41, no. 1, pp. 14–30, Jan. 2011.

[6] S. Rehberger, L. Spreiter, and B. Vogel-Heuser, "An agent-based approach for dependable planning of production sequences in automated production systems," *AT—Automatisierungstechnik*, vol. 65, no. 11, pp. 766–778, 2017.

[7] N. K. C. C. Krothapalli, and A. V. V. Deshmukh, "Design of negotiation protocols for multi-agent manufacturing systems," *Int. J. Prod. Res.*, vol. 37, no. 7, pp. 1601–1624, 1999.

[8] M. K. Lim, Z. Zhang, and W. T. Goh, "An iterative agent bidding mechanism for responsive manufacturing," *Eng. Appl. Artif. Intell.*, vol. 22, no. 7, pp. 1068–1079, 2009.

[9] Y. Sallez, T. Berger, and D. Trentesaux, "A stigmergic approach for dynamic routing of active products in FMS," *Comput. Ind.*, vol. 60, no. 3, pp. 204–216, 2009.

[10] C. Legat, D. Schütz, and B. Vogel-Heuser, "Automatic generation of field control strategies for supporting (re-)engineering of manufacturing systems," *J. Intell. Manuf.*, vol. 25, no. 5, pp. 1101–1111, 2014.

[11] W. Shen, "Distributed manufacturing scheduling using intelligent agents," *IEEE Intell. Syst.*, vol. 17, no. 1, pp. 88–94, Jan./Feb. 2002.

[12] S. Wang, J. Wan, D. Zhang, D. Li, and C. Zhang, "Towards smart factory for Industry 4.0: A self-organized multi-agent system with big data based feedback and coordination," *Comput. Netw.*, vol. 101, pp. 158–168, 2016.

[13] I. Kovalenko, K. Barton, and D. Tilbury, "Design and implementation of an intelligent product agent architecture in manufacturing systems," in *Proc. 22nd IEEE Int. Conf. Emerg. Technol. Factory Autom.*, 2007, pp. 1–8.

[14] I. Kovalenko, D. Tilbury, and K. Barton, "The model-based product agent: A control oriented architecture for intelligent products in multi-agent manufacturing systems," *Control Eng. Pract.*, vol. 86, no. Mar., pp. 105–117, 2019.

[15] B. R. Ferrer, B. Ahmad, A. Lobov, D. A. Vera, J. L. M. Lastra, and R. Harrison, "An approach for knowledge-driven product, process and resource mappings for assembly automation," in *Proc. IEEE Int. Conf. Autom. Sci. Eng.*, 2015, pp. 1104–1109.

[16] E. C. Balta, Y. Lin, K. Barton, D. M. Tilbury, and Z. M. Mao, "Production as a service: A digital manufacturing framework for optimizing utilization," *IEEE Trans. Autom. Sci. Eng.*, vol. 15, no. 4, pp. 1483–1493, Oct. 2018.

[17] D. McFarlane, S. Sarma, J. L. Chirn, C. Y. Wong, and K. Ashton, "Auto ID systems and intelligent manufacturing control," *Eng. Appl. Artif. Intell.*, vol. 16, no. 4, pp. 365–376, Jun. 2003.

[18] C. G. Cassandras and S. Lafortune, *Introduction to Discrete Event Systems*, 2nd ed. New Year, NY, USA: Springer, 2009.

[19] J. F. Pétin, D. Gouyon, and G. Morel, "Supervisory synthesis for product-driven automation and its application to a flexible assembly cell," *Control Eng. Pract.*, vol. 15, no. 5, pp. 595–614, 2007.

[20] "Industrie 4.0." 2015. [Online]. Available: http://i40d.ais.mw.tum.de/. Accessed on: Jun. 15, 2019.

[21] M. Hoffmann, "Adaptive and scalable information modeling to enable autonomous decision making for real-time interoperable factories," Ph.D. dissertation, Inst. Info. Manage. Mech. Eng., RWTH Aachen Univ., Aachen, Germany, 2017.

[22] Jade, "Java agent development framework." 2004. [Online]. Available: http://jade.tilab.com/

[23] Beckhoff Automation, "ADS, Automation Device Specification," 2019. [Online]. Available: www.beckhoff.com

[24] J. O'Madadhain, D. Fisher, S. White, and Y. Boey, "The JUNG (Java Universal Network/Graph) Framework," School Inf. Comput. Sci. Univ. California, Irvine, Irvine, CA, USA, UCI-ICS, Tech. Rep. UCI-ICS 03-17, Oct. 2003.